

西南交通大學
Southwest Jiaotong University

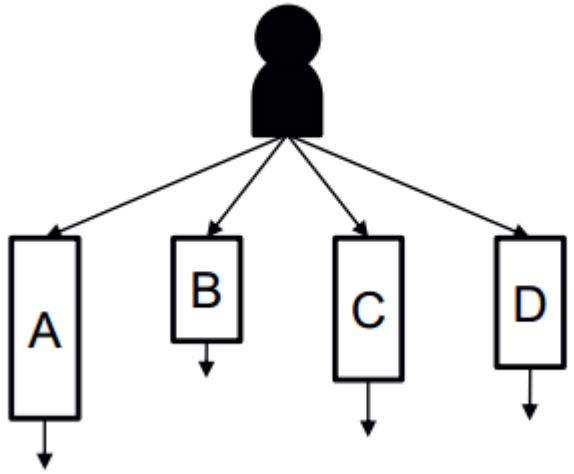
Operating Systems

Fei Teng
fteng@swjtu.edu.cn

Multiple Processes

- Operating System design is concerned with the management of processes and threads:
  - Multiprogramming
  - Multiprocessing
  - Distributed Processing
  - Multiuser

?



4.1 Principles of Concurrency （并发）

- Uniprocessor
  - Interleaving （交替）
  - Overlapping （重叠）
- Independent processes
  - Protect own resources (through different address spaces etc)
- Dependent processes
  - competition or cooperation
  - Provide for **concurrency**

Time	You	Housemate
15:00	Arrive home	
15:05	Look in fridge, no milk	
15:10	Leave for shop	
15:15		Arrive home
15:20	Arrive at shop	Look in fridge, no milk
15:25	Buy milk, leave	Leave for shop
15:30		
15:35	Arrive home	Arrive at shop
15:36	Put milk in fridge	
15:40		Buy milk, leave
15:45		
15:50		Arrive home
15:51		Put milk in fridge

The problem is this takes time and no coomunication

A programming example

```
int x = 0;
```
Shared between A and B

```
while(x < 10){
    x = x + 1;
}
fprintf("A wins");
```
A

```
while(x > -10){
    x = x - 1;
}
fprintf("B wins");
```
B

Two threads referencing x

1. Which one wins?
2. Is it guaranteed?

NON-DETERMINISTIC BEHAVIOUR

A programming example

- **Non-deterministic results depend on**
  - Activities of other processes
  - The way the OS handles interrupts
  - Scheduling policies of the OS

Context switch

Let X=1			
T0	Process A	X=X+1	X=2
T1	Process A	X=X+1	X=3
T2	Process B	X=X-1	X=2
T3	Process B	X=X-1	X=1
T4	Process B	X=X-1	X=0
T5	Process A	X=X+1	X=1
T6	Process B	X=X-1	X=0

5.8 Consider the following program:

```
P1: {
  shared int x;
  x = 10;
  while (1)  {
     x = x - 1;
     x = x + 1;
     if (x != 10)
       printf("x is %d",x)
     }
  }
}
```

```
P2:{
  shared int x;
  x = 10;
  while ( 1 ) {
     x = x - 1;
     x = x + 1;
     if (x!=10)
     printf("x is %d",x)
     }
  }
}
```

Note that the scheduler in a uniprocessor system would implement pseudo-parallel execution of these two concurrent processes by interleaving their instructions, without restriction on the order of the interleaving.

a. Show a sequence (i.e., trace the sequence of interleavings of statements) such that the statement "x is 10" is printed.

西南交通大學
Southwest Jiaotong University

Race Condition(竞争条件)

- Occurs when multiple processes or threads read and write data items
- The final result depends on the order of execution
  - the "loser" of the race is the process that updates last and will determine the final value of the variable

Operating System Concerns Concurrency

- Difficulties of Concurrency
  - Sharing of global resources
  - Difficult for the OS to manage the allocation of resources optimally
  - Difficult to locate programming errors as results are not deterministic and reproducible

- The OS must:
  - be able to keep track of various processes
  - allocate and de-allocate resources for each active process
  - protect the data and physical resources of each process against interference by other processes
  - ensure that the processes and outputs are independent of the processing speed

Interaction Process (交互)

- Processor
- Memory
- IO
- …

Degree of Awareness	Relationship	Influence that One Process Has on the Other	Potential Control Problems
Processes unaware of each other	Competition	•Results of one process independent of the action of others •Timing of process may be affected	•Mutual exclusion •Deadlock (renewable resource) •Starvation
Processes indirectly aware of each other (e.g., shared object)	Cooperation by sharing	•Results of one process may depend on information obtained from others •Timing of process may be affected	•Mutual exclusion •Deadlock (renewable resource) •Starvation •Data coherence
Processes directly aware of each other (have communication primitives available to them)	Cooperation by communication	•Results of one process may depend on information obtained from others •Timing of process may be affected	•Deadlock (consumable resource) •Starvation

Resource Competition

- Concurrent processes come into conflict when they are competing for use of the same resource
  - for example: I/O devices, memory, processor time, clock
- In the case of competing processes three control problems must be faced:
  - mutual exclusion
  - deadlock
  - starvation

Mutual Exclusion（互斥）

```
/* PROCESS 1 */

void P1
{
   while (true) {
      /* preceding code */;
      entercritical (Ra);
      /* critical section */;
      exitcritical (Ra);
      /* following code */;
   }
}
```

```
/* PROCESS 2 */

void P2
{
   while (true) {
      /* preceding code */;
      entercritical (Ra);
      /* critical section */;
      exitcritical (Ra);
      /* following code */;
   }
}
```

. . .

```
/* PROCESS n */

void Pn
{
   while (true) {
      /* preceding code */;
      entercritical (Ra);
      /* critical section */;
      exitcritical (Ra);
      /* following code */;
   }
}
```

Figure 5.1　Illustration of Mutual Exclusion

Requirements for Mutual Exclusion

- Must be enforced
- A process that halts must do so without interfering with other processes
- No deadlock or starvation
- A process must not be denied access to a critical section when there is no other process using it （进步）
- No assumptions are made about relative process speeds or number of processes
- A process remains inside its critical section for a finite time （有限等待）

Petersons Algorithm

Operation:

- Two threads of alternating execution, i = 0 and j = 1
- Turn: Whose turn is it?
- Flag: Want to enter critical section
- Spinlock (**Busy Waiting**)

Satisfies:

- Mutual exclusion
- Progress
- Bounded Waiting

```
int turn;
bool flag[2] = 0;

do {
    flag[i] = TRUE;
    turn = j;
    while (flag[j] && turn == j);

    CRITICAL_SECTION

    flag[i] = FALSE;

    REMAINDER_SECTION
} while(TRUE);
```

Mutual Exclusion Tools

- Hardware support
  - Interrupt (uniprocesor)
  - Machine instruction (MPS)
- Software support
  - Semaphore（信号量）
  - Monitor
  - message

Mutual Exclusion: Hardware Support

- Interrupt Disabling（中断禁用）
  - uniprocessor system
  - disabling interrupts guarantees mutual exclusion
- Disadvantages:
  - the efficiency of execution could be noticeably degraded
  - this approach will not work in a multiprocessor architecture

Mutual Exclusion: Hardware Support

- Special Machine Instructions
  - Compare&Swap Instruction
    - ✓ also called a “compare and exchange instruction”
    - ✓ a compare is made between a memory value and a test value
    - ✓ if the values are the same a swap occurs
    - ✓ carried out atomically

Compare and Swap Instruction

```
/* program mutualexclusion */
const int n = /* number of processes */;
int bolt;
void P(int i)
{
   while (true) {
     while (compare_and_swap(bolt, 0, 1) == 1)
          /* do nothing */;
       /* critical section */;
       bolt = 0;
       /* remainder */;
   }
}
void main()
{
   bolt = 0;
   parbegin (P(1), P(2), ... ,P(n));

}
```

(a) Compare and swap instruction

Figure 5.2 Hardware Support for Mutual Exclusion

西南交通大學 Southwest Jiaotong University

Exchange Instruction

```
    /* program mutualexclusion */
int const n = /* number of processes**/;
int bolt;
void P(int i)
{
    int keyi = 1;
    while (true) {
        do exchange (keyi, bolt)
        while (keyi != 0);
        /* critical section */;
        bolt = 0;
        /* remainder */;
    }
}
void main()
{
    bolt = 0;
    parbegin (P(1), P(2), ..., P(n));
}
```

(b) Exchange instruction

西南交通大學 Southwest Jiaotong University

Figure 5.2 Hardware Support for Mutual Exclusion

Special Machine Instruction

Advantages

- Applicable to any number of processes on either a single processor or multiple processors sharing main memory
- Simple and easy to verify
- It can be used to support multiple critical sections; each critical section can be defined by its own variable

Disadvantages

- Busy-waiting（忙等待） is employed, thus while a process is waiting for access to a critical section it continues to consume processor time
- Starvation（饥饿） is possible when a process leaves a critical section and more than one process is waiting
- Deadlock（死锁） is possible

4.3 Semaphore

- A variable that has an integer value upon which only three operations are defined:
  - May be initialized to a nonnegative integer value
  - The semWait operation decrements the value
  - The semSignal operation increments the value



- Limitation
  - There is no way to know before a process decrements a semaphore whether it will block or not
  - There is no way to know which process will continue immediately on a uniprocessor system when two processes are running concurrently
  - You don't know whether another process is waiting so the number of unblocked processes may be zero or one

Semaphore Primitives

```
struct semaphore {
      int count;
      queueType queue;
};
void semWait(semaphore s)
{
      s.count--;
      if (s.count < 0) {
            /* place this process in s.queue */;
            /* block this process */;
      }
}
void semSignal(semaphore s)
{
      s.count++;
      if (s.count <= 0) {
            /* remove a process P from s.queue */;
            /* place process P on ready list */;
      }
}
```

Figure 5.3 A Definition of Semaphore Primitives

西南交通大學 Southwest Jiaotong University

Binary Semaphore Primitives

```
struct binary_semaphore {
      enum {zero, one} value;
      queueType queue;
};
void semWaitB(binary_semaphore s)
{
      if (s.value == one)
            s.value = zero;
      else {
               /* place this process in s.queue */;
               /* block this process */;
      }
}
void semSignalB(semaphore s)
{
      if (s.queue is empty())
            s.value = one;
      else {
               /* remove a process P from s.queue */;
               /* place process P on ready list */;
      }
}
```

Figure 5.4 A Definition of Binary Semaphore Primitives

西南交通大學 Southwest Jiaotong University

Strong/Weak Semaphores

- A queue is used to hold processes waiting on the semaphore
- Strong Semaphores
  - the process that has been blocked the longest is released from the queue first (FIFO)
- Weak Semaphores
  - the order in which processes are removed from the queue is not specified

Example of Semaphore Mechanism

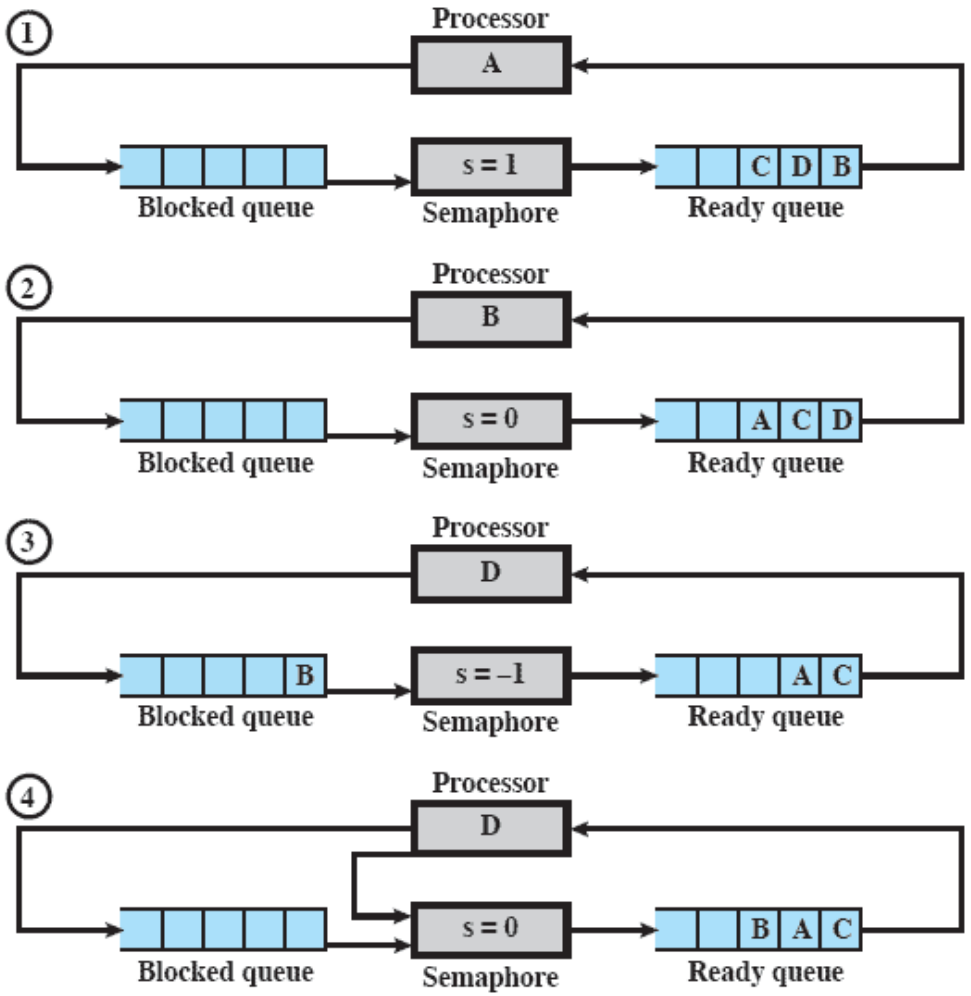


Figure 5.5 Example of Semaphore Mechanism

西南交通大學
Southwest Jiaotong University

Mutual Exclusion

```
/* program mutualexclusion */
const int n = /* number of processes  */;
semaphore s = 1;
void P(int i)
{
     while (true) {
          semWait(s);
          /* critical section   */;
          semSignal(s);
          /* remainder   */;
     }
}
void main()
{
     parbegin (P(1), P(2), . . ., P(n));
}
```

Figure 5.6 Mutual Exclusion Using Semaphores

西南交通大學
Southwest Jiaotong University

Shared Data Protected by a Semaphore

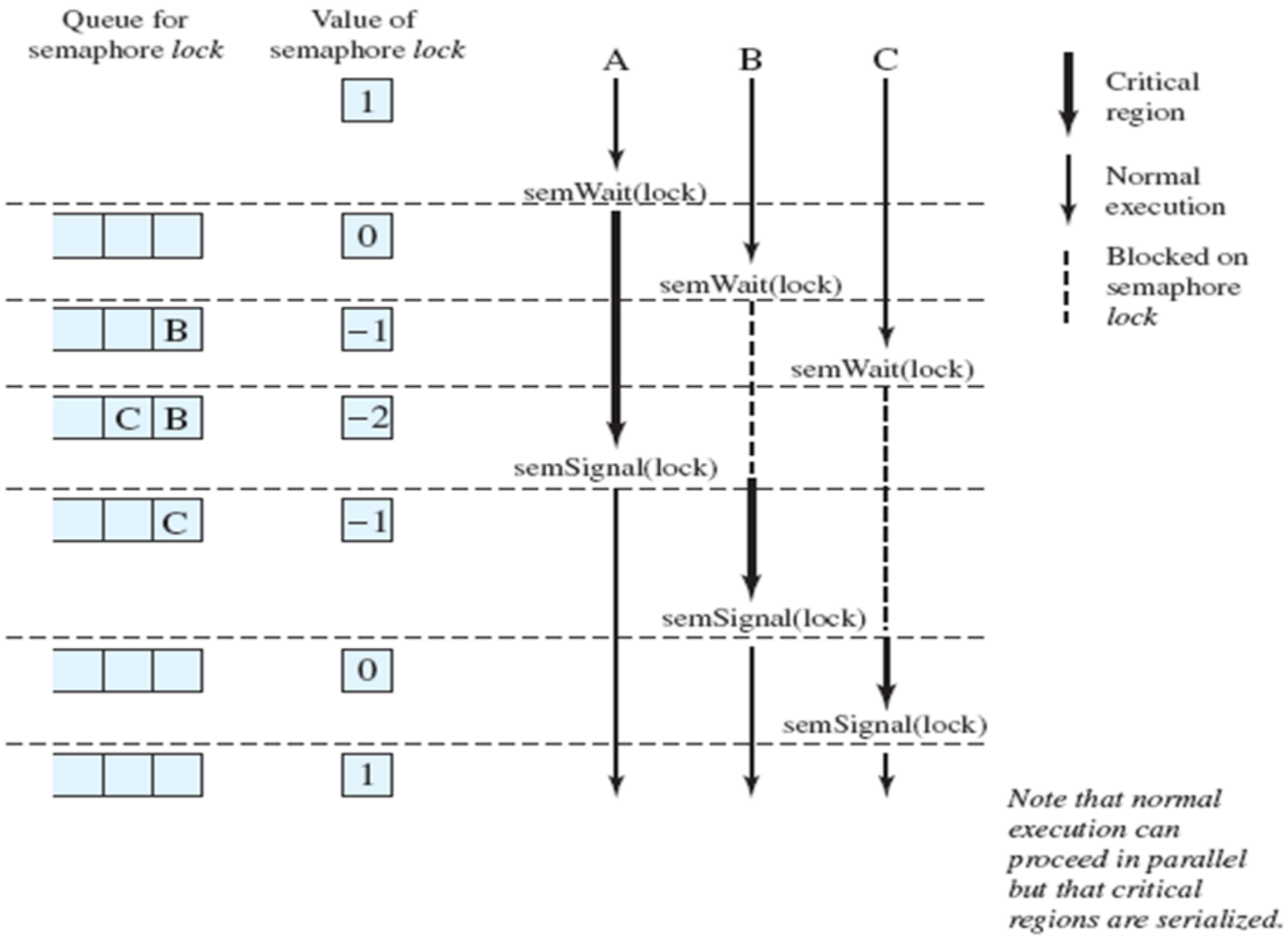


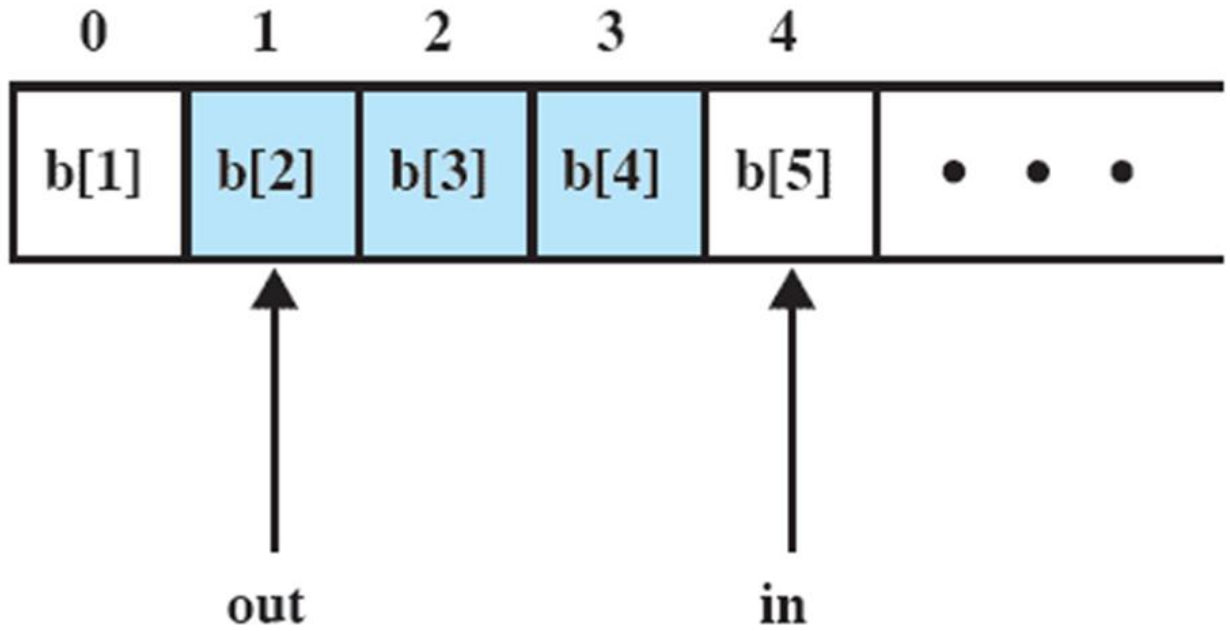
Figure 5.7 Processes Accessing Shared Data Protected by a Semaphore

西南交通大學 Southwest Jiaotong University

Producer/Consumer Problem

- Situation:
  - one or more producers are generating data and placing these in a buffer
  - a single consumer is taking items out of the buffer one at time
  - only one producer or consumer may access the buffer at any one time
- Problem:.
  - Mutual exclusion-one process visit the buffer
  - Synchronization（同步）- producer can’t add data into full buffer
  - Synchronization- consumer can’t remove data from an empty buffer

Buffer Structure



Note: shaded area indicates portion of buffer that is occupied

Figure 5.8 Infinite Buffer for the Producer/Consumer Problem

西南交通大学
Southwest Jiaotong University

A Solution

```
/* program producerconsumer */
int n;
binary_semaphore s = 1, delay = 0;
void producer()
{
     while (true) {
          produce();
          semWaitB(s);
          append();
          n++;
          if (n==1) semSignalB(delay);
          semSignalB(s);
     }
}
void consumer()
{
     semWaitB(delay);
     while (true) {
          semWaitB(s);
          take();
          n--;
          semSignalB(s);
          consume();
          if (n==0) semWaitB(delay);
     }
}
void main()
{
     n = 0;
     parbegin (producer, consumer);
}
```

西南交通大学
Southwest Jiaotong University

Table 5.4 Possible Scenario for the Program of Figure 5.9

	Producer	Consumer	s	n	Delay
1			1	0	0
2	semWaitB(s)		0	0	0
3	n++		0	1	0
4	**if** (n==1) (semSignalB(delay))				
5	semSignalB(s)				
6		semWaitB(delay)			
7		semWaitB(s)			
8		n--			
9		semSignalB(s)			
10	semWaitB(s)				
11	n++				
12	**if** (n==1) (semSignalB(delay))				
13	semSignalB(s)				
14		**if** (n==0) (semWaitB(delay))			
15		semWaitB(s)			
16		n--			
17		semSignalB(s)			
18		**if** (n==0) (semWaitB(delay))			
19		semWaitB(s)			
20		n--			
21		semiSignlaB(s)			

NOTE: White areas represent the critical section controlled by semaphore s.

西南交通大學 Southwest Jiaotong University

Table 5.4 Possible Scenario for the Program of Figure 5.9

	Producer	Consumer	s	n	Delay
1			1	0	0
2	semWaitB(s)		0	0	0
3	n++		0	1	0
4	**if** (n==1) (semSignalB(delay))		0	1	1
5	semSignalB(s)		1	1	1
6		semWaitB(delay)	1	1	0
7		semWaitB(s)	0	1	0
8		n--	0	0	0
9		semSignalB(s)	1	0	0
10	semWaitB(s)		0	0	0
11	n++		0	1	0
12	**if** (n==1) (semSignalB(delay))		0	1	1
13	semSignalB(s)		1	1	1
14		**if** (n==0) (semWaitB(delay))	1	1	1
15		semWaitB(s)	0	1	1
16		n--	0	0	1
17		semSignalB(s)	1	0	1
18		**if** (n==0) (semWaitB(delay))	1	0	0
19		semWaitB(s)	0	0	0
20		n--	0	–1	0
21		semiSignlaB(s)	1	–1	0

NOTE: White areas represent the critical section controlled by semaphore s.

?

西南交通大學 Southwest Jiaotong University

Correct Solution

```
/* program producerconsumer */
int n;
binary_semaphore s = 1, delay = 0;
void producer()
{
     while (true) {
          produce();
          semWaitB(s);
          append();
          n++;
          if (n==1) semSignalB(delay);
          semSignalB(s);
     }
}
void consumer()
{
     int m; /* a local variable */
     semWaitB(delay);
     while (true)  {
          semWaitB(s);
          take();
          n--;
          m = n;
          semSignalB(s);
          consume();
          if (m==0) semWaitB(delay);
     }
}
void main()
{
     n = 0;
     parbegin (producer, consumer);
}
```

```
/* program  producerconsumer */
semaphore n = 0, s = 1;
void producer()
{
     while (true) {
          produce();
          semWait(s);
          append();
          semSignal(s);
          semSignal(n);
     }
}
void consumer()
{
     while (true) {
          semWait(n);
          semWait(s);
          take();
          semSignal(s);
          consume();
     }
}
void main()
{
     parbegin (producer, consumer);
}
```

?

Figure 5.11 A Solution to the Infinite-Buffer Producer/Consumer Problem Using Semaphores

西南交通大學 Southwest Jiaotong University

Finite Circular Buffer

Block on:	Unblock on:
Producer: insert in full buffer	Consumer: item inserted
Consumer: remove from empty buffer	Producer: item removed

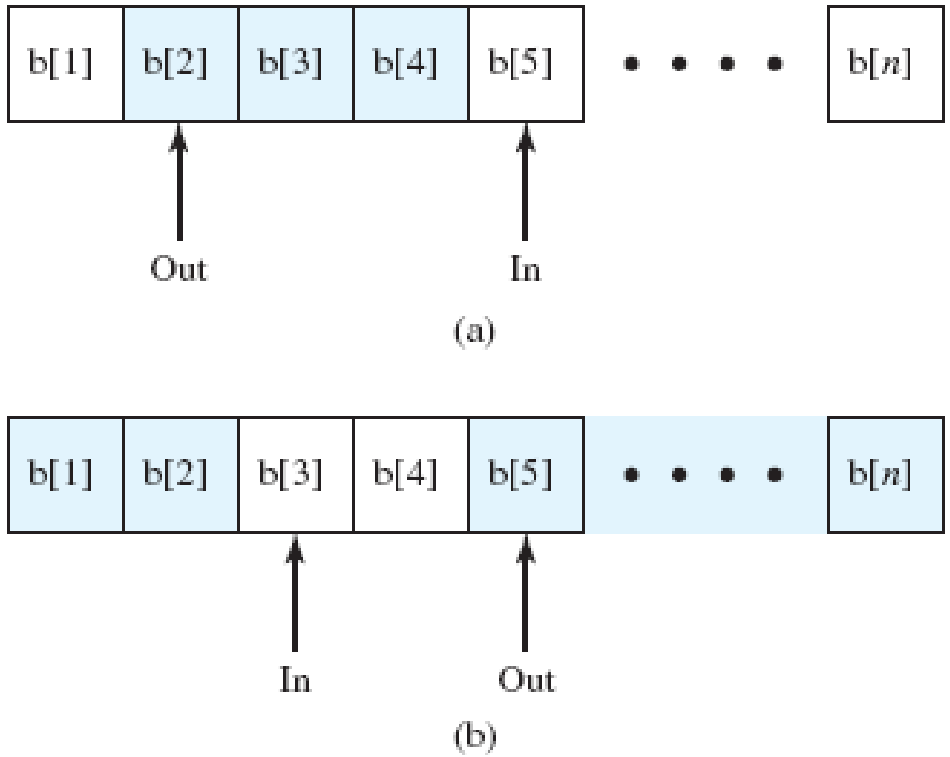


Figure 5.12 Finite Circular Buffer for the Producer/Consumer Problem

西南交通大学 Southwest Jiaotong University

```
/* program boundedbuffer */
const int sizeofbuffer = /* buffer size */;
semaphore s = 1, n= 0, e= sizeofbuffer;
void producer()
{
     while (true) {
          produce();
          semWait(e);
          semWait(s);
          append();
          semSignal(s);
          semSignal(n);
     }
}
void consumer()
{
     while (true) {
          semWait(n);
          semWait(s);
          take();
          semSignal(s);
          semSignal(e);
          consume();
     }
}
void main()
{
     parbegin (producer, consumer);
}
```



Readers/Writers Problem

- A data area is shared among many processes
  - some processes only read the data area (readers)
  - some only write to the data area (writers)

- Conditions that must be satisfied:
  - any number of readers may simultaneously read the file
  - only one writer at a time may write to the file
  - if a writer is writing to the file, no reader may read it

Readers Have Priority

```
/* program readersandwriters */
int readcount;
semaphore x = 1, wsem = 1;
void reader()
{
    while (true) {
      semWait (x);
      readcount++;
      if (readcount == 1) semWait (wsem);
      semSignal (x);
      READUNIT();
      semWait (x);
      readcount--;
      if (readcount == 0) semSignal (wsem);
      semSignal (x);
    }
 }
void writer()
{
    while (true) {
      semWait (wsem);
      WRITEUNIT();
      semSignal (wsem);
    }
}

void main()
{
    readcount = 0;
    parbegin (reader, writer);
}
```

X -> readcount

Writers Have Priority

```
/*program readersandwriters*/
int  readcount, writecount;
semaphore x = 1, y = 1, z = 1, wsem = 1, rsem = 1;
void reader()
{
    while (true) {
     semWait (z);
          semWait (rsem);
               semWait (x);
                    readcount++;
                    if (readcount == 1) semWait (wsem);
               semSignal (x);
          semSignal (rsem);
     semSignal (z);
     READUNIT();
     semWait (x);
          readcount--;
          if (readcount == 0) semSignal (wsem);
     semSignal (x);
    }
}
```

z-> rsem

x-> readcount

```
void writer ()
{
    while (true) {
     semWait (y);
          writecount++;
          if (writecount == 1) semWait (rsem);
     semSignal (y);
     semWait (wsem);
     WRITEUNIT();
     semSignal (wsem);
     semWait (y);
          writecount--;
          if (writecount == 0) semSignal (rsem);
     semSignal (y);
    }
}
void main()
{
    readcount = writecount = 0;
    parbegin (reader, writer);
}
```

y -> writecount

- (Readers/Writers problem) At 1:00pm, a writer W1 begins writing. At 1:30pm, the reader R1 arrives and tries to read, but fails because W1 is writing. At 2:00pm, readers R2, R3 and R4 arrive. The writer W1 finishes writing and leaves at 3:00pm. Each reading / writing operation takes 2 hours.
- Trace what happens to the readers when they arrive (e.g. where they are blocked), and how all the readers are unblocked at 3:00pm when the writer finishes writing. Trace the readers until all of them are reading.

```
    /* reader thread */
    repeat
a1    wait(x);
a2      readcount = readcount+1;
a3      if (readcount==1) wait(wsem);
a4    signal(x);
a5    READ;
a6    wait(x);
        readcount = readcount-1;
        if (readcount==0) signal(wsem);
      signal(x);
    forever;
```

```
    /* writer thread */
    repeat
b1    wait(wsem);
b2      WRITE;
b3    signal(wsem);
    forever
```

西南交通大學 Southwest Jiaotong University

			Readcount	X (信号量)	wsem	
			0	1	1	
1:00	W1	b1	0	1	0	
		b2				WRITE file
1:30	R1	a1	0	0	0	
		a2	1			
		a3			-1	Blocked
2:00	R2					
	R3					
	R4					
3:00	W1					
	R1					
	R1					
	R2					
	R2					
	R2					
	R2					
	R3					
	R3					
	R3					
	R3					
	R4					
	R4					
	R4					
	R4					

西南交通大學
Southwest Jiaotong University

Monitors

- Programming language construct(程序语言结构) that provides equivalent functionality to that of semaphores and is easier to control
- Implemented in many programming languages
  - including Concurrent Pascal, Pascal-Plus, Modula-2, Modula-3, and Java
- Software module consisting of one or more procedures, an initialization sequence, and local data

Monitor Characteristics

- Local data variables are accessible only by the monitor's procedures （过程）
  and not by any external procedure
- Process enters monitor by invoking one of its procedures
- Only one process may be executing in the monitor at a time -> Mutual exclusion

Synchronization

- Achieved by the use of condition variables （条件变量） that are contained within the monitor and accessible only within the monitor
  - Condition variables are operated on by two functions:
  - cwait(c): suspend execution of the calling process on condition c
  - csignal(c): resume execution of some process blocked after a cwait on the same condition

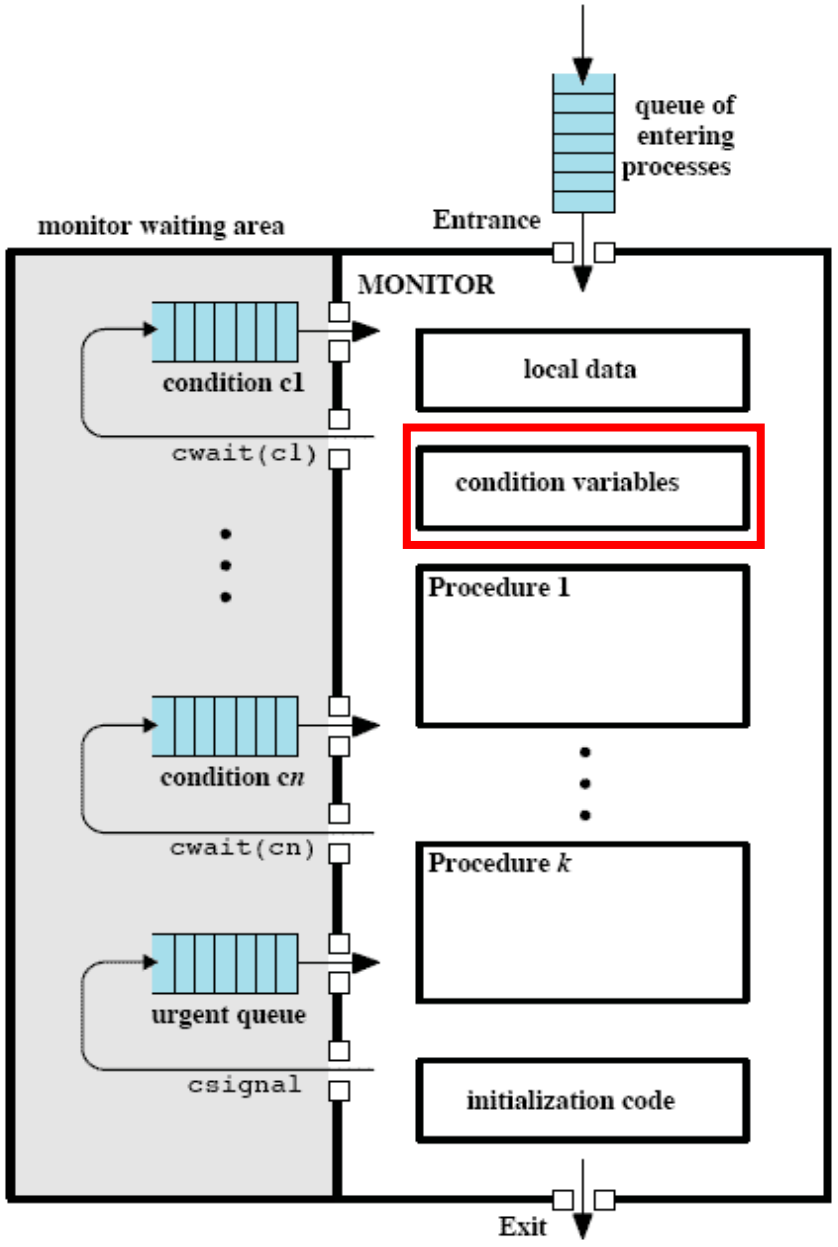


Figure 5.15 Structure of a Monitor

Problem Solution Using a Monitor

```
/* program producerconsumer */
monitor boundedbuffer;
char buffer [N];                                    /* space for N items */
int nextin, nextout;                                /* buffer pointers */
int count;                                          /* number of items in buffer */
cond notfull, notempty;          /* condition variables for synchronization */

void append (char x)
{
     if (count == N) cwait(notfull);       /* buffer is full; avoid overflow */
     buffer[nextin] = x;
     nextin = (nextin + 1) % N;
     count++;
     /* one more item in buffer */
     csignal(notempty);                        /* resume any waiting consumer */
}
void take (char x)
{
     if (count == 0) cwait(notempty);    /* buffer is empty; avoid underflow */
     x = buffer[nextout];
     nextout = (nextout + 1) % N;
     count--;                                    /* one fewer item in buffer */
     csignal(notfull);                         /* resume any waiting producer */
}
{                                                          /* monitor body */
     nextin = 0; nextout = 0; count = 0;           /* buffer initially empty */
}
```

```
void producer()
{
     char x;
     while (true) {
     produce(x);
     append(x);
     }
}
void consumer()
{
     char x;
     while (true) {
      take(x);
       consume(x);
     }
}
void main()
{
     parbegin (producer, consumer);
}
```

Can the Producer/consumer visit buffer directly?

西南交通大学 Southwest Jiaotong University

```
/* program boundedbuffer */
const int sizeofbuffer = /* buffer size */;
semaphore s = 1, n= 0, e= sizeofbuffer;
void producer()
{
     while (true) {
          produce();
          semWait(e);
          semWait(s);
          append();
          semSignal(s);
          semSignal(n);
     }
}
void consumer()
{
     while (true) {
          semWait(n);
          semWait(s);
          take();
          semSignal(s);
          semSignal(e);
          consume();
     }
}
void main()
{
     parbegin (producer, consumer);
}
```



Message Passing

- When processes interact with one another two fundamental requirements must be satisfied:
  - Synchronization: to enforce mutual exclusion
  - Communication: to exchange information

- Message Passing is one approach to providing both of these functions
  - works with distributed systems and shared memory multiprocessor and uniprocessor systems

Message Passing

- The actual function is normally provided in the form of a pair of primitives (原语):
  - send (destination, message)
  - receive (source, message)
- A process sends information in the form of a message to another process designated by a destination
- A process receives information by executing the receive primitive, indicating the source and the message

Message Passing

Synchronization	**Format**
Send	Content
blocking	Length
nonblocking	fixed
Receive	variable
blocking	
nonblocking	**Queueing Discipline**
test for arrival	FIFO
	Priority
Addressing	
Direct	
send	
receive	
explicit	
implicit	
Indirect	
static	
dynamic	
ownership	

Table 5.5 Design Characteristics of Message Systems for Interprocess Communication and Synchronization

西南交通大学 Southwest Jiaotong University

Synchronization

- Communication of a message between two processes implies synchronization between the two
  - the receiver cannot receive a message until it has been sent by another process
- When a send primitive is executed in a process there are two possibilities:
  - The sending process is blocked until the message is received
  - The sending process is not blocked
- When a receive primitive is executed in a process there are two possibilities:
  - if there is no waiting message the process is blocked until a message arrives or the process continues to execute, abandoning the attempt to receive
  - if a message has previously been sent the message is received and execution continues

Blocking Send, Blocking Receive

- Both sender and receiver are blocked until the message is delivered
- Sometimes referred to as a rendezvous（会合）
- Allows for tight synchronization between processes

Nonblocking Send

- Nonblocking send, blocking receive
  - sender continues on but receiver is blocked until the requested message arrives
  - most useful combination
  - sends one or more messages to a variety of destinations as quickly as possible
  - example -- a service process that exists to provide a service or resource to other processes
- Nonblocking send, nonblocking receive
  - neither party is required to wait

Addressing（寻址）

Direct addressing

- Send primitive includes a specific identifier of the destination process
- Receive primitive can be handled :
  - ✓ require that the process explicitly designate a sending process, effective for cooperating concurrent processes
  - ✓ implicit addressing, source parameter of the receive primitive possesses a value returned when the receive operation has been performed

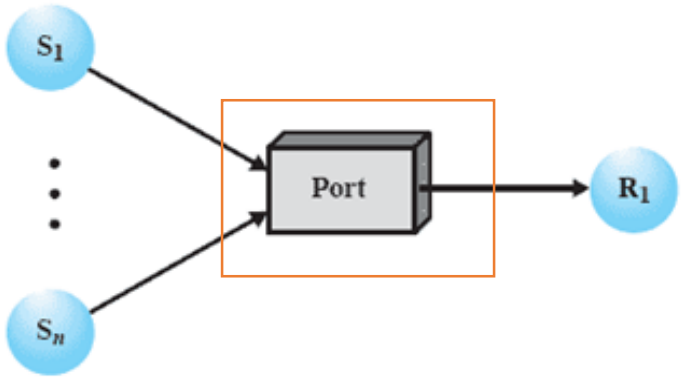
Indirect addressing

- Messages are sent to a shared data structure consisting of queues that can temporarily hold messages
- Queues are referred to as mailboxes
  - ✓ One process sends a message to the mailbox and the other process picks up the message from the mailbox
- Allows for greater flexibility in the use of messages

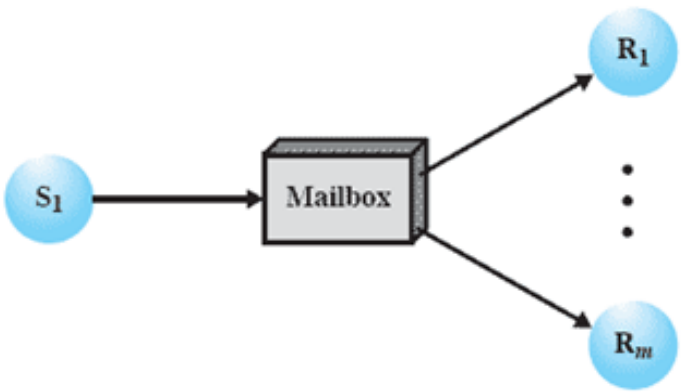
Indirect Process



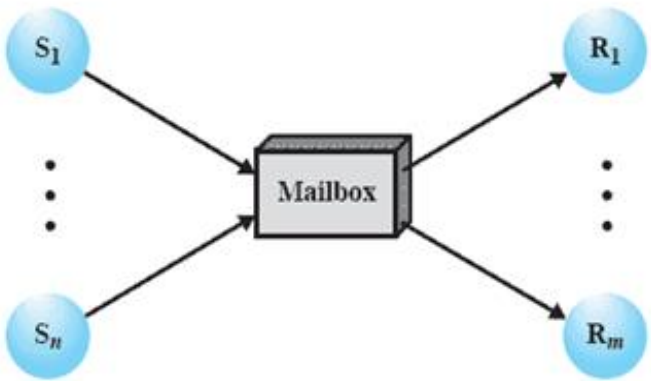
(a) One to one



(b) Many to one



(c) One to many



(d) Many to many

西南交通大學
Southwest Jiaotong University

General Message Format

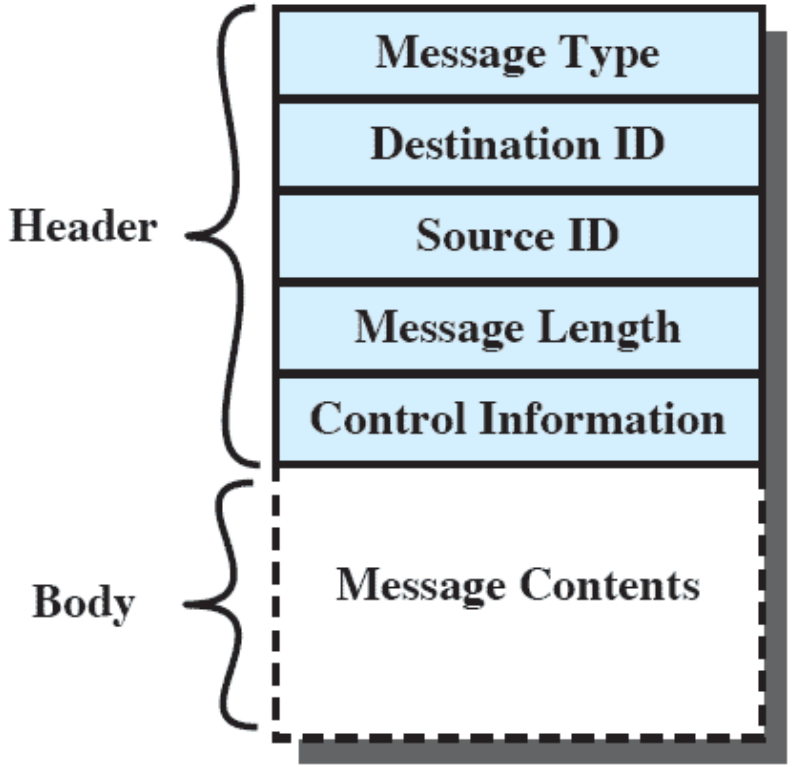


Figure 5.19 General Message Format

西南交通大學
Southwest Jiaotong University

Mutual Exclusion

```
/* program mutualexclusion */
const int n = /* number of processes  */;
void P(int i)
{
    message msg;
    while (true) {
     receive (box, msg);
     /* critical section   */;
     send (box, msg);
     /* remainder   */;
    }
}
void main()
{
    create mailbox (box);
    send (box, null);
    parbegin (P(1), P(2), . . ., P(n));
}
```

Figure 5.20 Mutual Exclusion Using Messages

西南交通大學
Southwest Jiaotong University

Message Passing Example

```
const int
    capacity = /* buffering capacity */ ;
    null =/* empty message */ ;
int i;
void producer()
{   message pmsg;
    while (true) {
     receive (mayproduce, pmsg);
     pmsg = produce();
     send (mayconsume, pmsg);
    }
}
void consumer()
{   message cmsg;
    while (true) {
     receive (mayconsume, cmsg);
     consume (cmsg);
     send (mayproduce, null);
    }
}

void main()
{
    create_mailbox (mayproduce);
    create_mailbox (mayconsume);
    for (int i = 1; i <= capacity; i++) send (mayproduce, null);
    parbegin (producer, consumer);
}
```

May used in distributed system with multiple producers and consumers

Some Key Terms Related to Concurrency

atomic operation	A function or action implemented as a sequence of one or more instructions that appears to be indivisible; that is, no other process can see an intermediate state or interrupt the operation. The sequence of instruction is guaranteed to execute as a group, or not execute at all, having no visible effect on system state. Atomicity guarantees isolation from concurrent processes.
critical section	A section of code within a process that requires access to shared resources and that must not be executed while another process is in a corresponding section of code.
deadlock	A situation in which two or more processes are unable to proceed because each is waiting for one of the others to do something.
livelock	A situation in which two or more processes continuously change their states in response to changes in the other process(es) without doing any useful work.
mutual exclusion	The requirement that when one process is in a critical section that accesses shared resources, no other process may be in a critical section that accesses any of those shared resources.
race condition	A situation in which multiple threads or processes read and write a shared data item and the final result depends on the relative timing of their execution.
starvation	A situation in which a runnable process is overlooked indefinitely by the scheduler; although it is able to proceed, it is never chosen.

Summary

- Messages
  - Useful for the enforcement of mutual exclusion discipline
- Operating system themes are:
  - Multiprogramming, multiprocessing, distributed processing
  - Fundamental to these themes is concurrency
    - ✓ issues of conflict resolution and cooperation arise
- Mutual Exclusion
  - Condition in which there is a set of concurrent processes, only one of which is able to access a given resource or perform a given function at any time
  - One approach involves the use of special purpose machine instructions
- Semaphores
  - Used for signaling among processes and can be readily used to enforce a mutual exclusion discipline